Chain of Custody: Technical Write-up

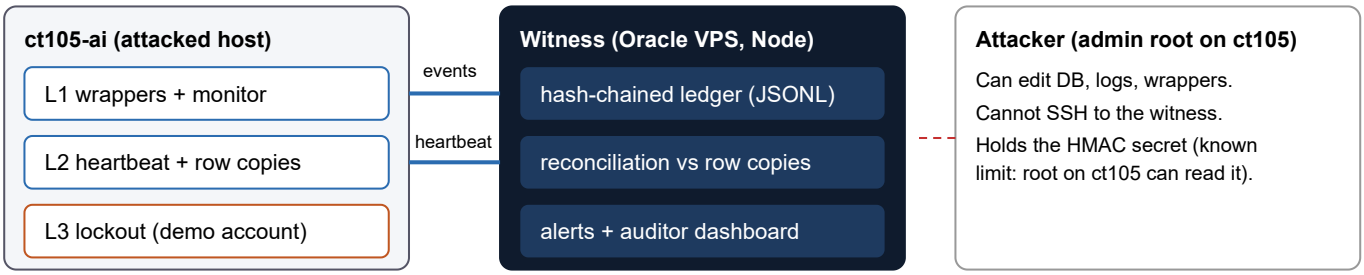
Hackurity 2026, PS-6 (Track C). Firewall Crew: Jeevika, Jerush, Jaganath. Status key: **working** = demonstrated in code and tests; **partial** = some of it works; **not done** = not built.

1. Problem and scope

Anyone with admin root on a log host can edit, truncate or silence the logs, and a SIEM that stores what it receives cannot tell. Logs used as evidence (banking, audit, incident response) then prove nothing. We protect one banking application running on ct105-ai, with a separate witness service on an Oracle VPS that keeps its own copy of the evidence. In scope: the attacks in the coverage map (section 2). Out of scope: the SIEM vendor, production hardening, and edits made before the first ship.

2. Approach and architecture

Assume each layer can fall; the next still catches the attack. Alerts appear only on the witness, never on the attacked host.



Layer 1 (**partial**) detects raw edits to the log path by wrapping the commands an admin would use (rm, sed, vi, truncate) and forwards each attempt to the witness. It does not block. Layer 2 (**working**) has two parts: a heartbeat that raises a silence alert after missed beats, and a SHA-256 hash chain on the witness copy that rejects a modified ledger. Bank rows are compared against the witness's copies. Layer 3 (**partial**) locks a single demo account; a full root lockout is built but gated and not enabled.

Coverage claim (attack)	Status	What is built, and where to see it
Raw rm / sed / truncate on the log	**Partial**	Detection only; wrappers log the attempt and the command still runs. `deploy/ct105/layer1/`. Not built: command allowlist, append-only attribute (chattr +a).
Kill the agent / network cut (silence)	**Working**	30 s heartbeat (15 to 120 s adaptive); the witness alerts after missed beats. `witness/server.js`, heartbeat tests in `witness/test/`. Not built: random jitter, per-beat nonce.
Edit a committed record	**Working**	SHA-256 prev_hash chain; a modified record is rejected on startup (`store.test.js`). Not built: per-entry HMAC tag and forward-secure key ratchet.
Change amount / type / status in the bank DB	**Working**	Reconciliation against witness row copies, with bounded random rechecks (`witness/reconcile.js`, `bank-recheck.test.js`). Demo: `deploy/ct105/layer2/attack-layer2.sh`, reversible with `restore-layer2.sh`.
Confirmed tampering (lockout)	**Partial**	Demo mode locks one account and unlocks with 2 of 3 demo keys (hash-checked, not Shamir). Full root lockout: built, gated, not enabled on ct105. `deploy/ct105/layer3/`.
Seal on silence	**Not done**	Witness quarantine on a broken link runs as a dry run only.
Auditor dashboard	**Working**	Shows layer events, heartbeats, reconciliation, and "Restore to normal". `witness/auditor.html`, served at the witness root.

4. Stretch objectives

We claim no stretch objective as working. The pitch deck describes a command gate, a 2-of-3 Shamir unlock, forward-secure keys and nonce heartbeats. These are the target design; none of them is built, and none is shown in the demo. The cybersec detector UI is served through the witness (**partial**), but its upstream repository has no licence, so it is not redistributed here without permission.

5. Threat and user model

Attacker: admin root on ct105 (edits the database, the logs, the wrappers, the monitor), with no shell on the witness. User: a SOC analyst or auditor who reads the witness dashboard and trusts it because the witness keeps the copy the attacker cannot change.

6. Assumptions

- The witness host is not compromised and its clock is correct.
- The HMAC secret shared with ct105 stays confidential. Root on ct105 can read it, so a root attacker can forge new events. This is stated, not solved.
- ct105 sends the witness its rows and heartbeats; an edit made before the first send is not caught (a known limit, section 8).

7. Evidence

- Witness unit tests: 33 of 33 pass (`node --test`, Node 24.15.0, 15 Oct 2026). Covers the hash chain, startup rejection, reconciliation, heartbeat timing and clamping, and back-off.
- Heartbeat interval: default 30 s, clamped to 15 to 120 s (unit test).
- Layer 2 demo: three attacks (amount, type, status). Timings in the demo script comments are design targets (about 10 s for the four-field check, the next 30 s heartbeat for status). **We have not measured detection latency end to end.**
- Python tests under `deploy/ct105/` (Layer 1, Layer 2, Layer 3) exist but were **not run** for this write-up.
- No false-positive rate or load test has been measured.

8. Known limitations and next steps

- Layer 1 detects but does not block. Next: an allowlist wrapper and an append-only attribute on the log file.
- The HMAC secret is shared with ct105, so root on ct105 can forge events. Next: per-entry signing with forward-secure keys.
- An edit before the first ship is not caught. Next: witness the outbox copy at write time.
- Full Layer 3 lockout is not enabled on ct105. It needs a non-root monitor identity first.
- The pitch deck's coverage map and architecture show the target design. The build differs as listed in section 3, and the demo shows only what is marked working or partial.
- Not yet verified on a clean machine (the README run steps are untested by a teammate).

AI tools used

Claude Code (Anthropic): code, tests and documentation. GitHub Copilot: code completion. Commit trailers show Claude Haiku 5.5, Claude Sonnet 5 and Claude Sonnet 5.5. The team designed the architecture and can explain every part.